Experiment 6 I^2C Communication

ART 11/11/26

In the Lab

1. Start the host GUI & UART link. Extend your Lab8 host application or create a new GUI with serial-port communication & raw hex transmission. Add UART to the working master project. From your GUI, send 24 49 00 & verify 24 49 02 33 75. Handle responses that arrive in multiple serial reads.

Communication History

```
2026-11-12 11:11:06.074 | COM5 | TX | LEN=3 | 24 49 00 | Auto identity request
2026-11-12 11:11:06.135 | COM5 | RX | LEN=5 | 24 49 02 33 75 | Identity discovery response
2026-11-12 11:11:06.157 | COM5 | INFO | LEN=0 |  | Verified bridge connected at COM5
```

Figure 4-2 GUI Send 244900, Received 24 49 02 33 75

Figure 4-2 ~~GUI Send~~ 244900 PIC & ADS1115 Responses ART 11/12/26

2. Address Probes. Implement probe command 50. Probe the PIC slave at 20 & an address you have confirmed is unoccupied. Confirm ACK/NACK with decoded start, address, & Stop on the scope

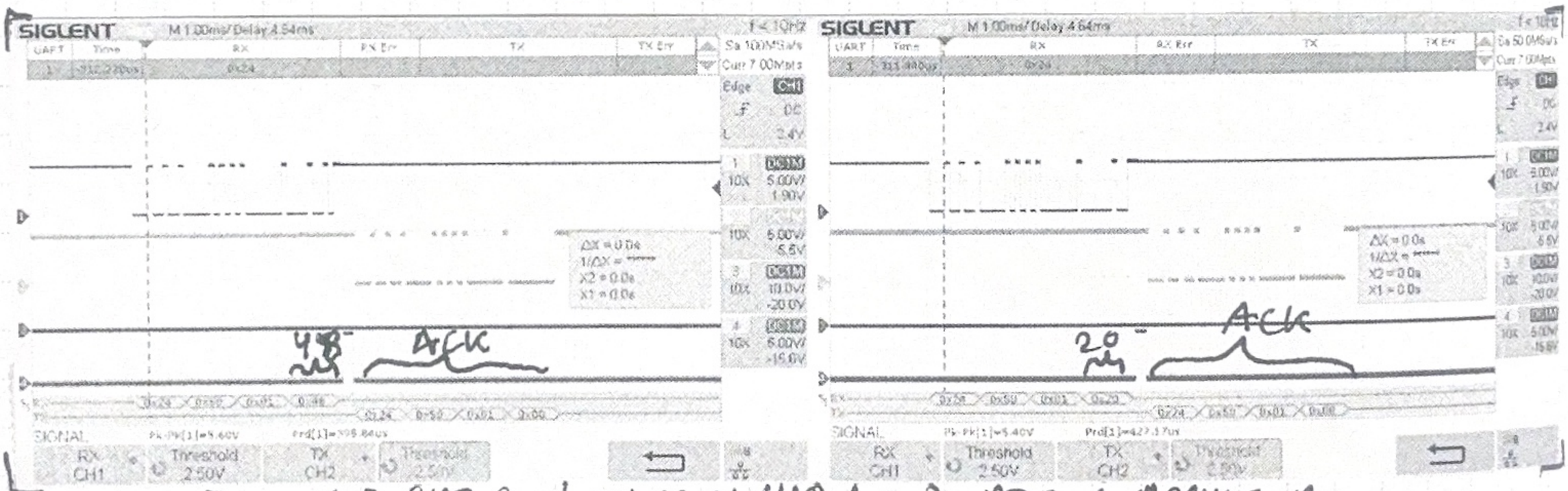


Figure 4-3 GUI Send 24 50 01 (48 & 20) PIC & ADS1115 Responses

3. Generic write/read. ~~Implemented probe command 50. Probe the PIC slave at 20~~ Implement write command 57 & read command 52 using your existing I^2C transaction routines & bounded buffers. Send 24 57 03 20 01 41 to display A. Write pointer 0x00, read PortB & change a switch. Write Pointer 0x02, read the two byte PIC ADC & compare with the potentiometer voltage.

Communication History

max POT → 2026-11-12 12:03:29.603 | COM5 | TX | LEN=5 | 24 52 02 20 02 | Manual I2C read 0x20, count 2
2026-11-12 12:03:29.635 | COM5 | RX | LEN=6 | 24 52 03 00 03 FF | I2C read successful: 03 FF | PIC AI
2026-11-12 12:03:36.911 | COM5 | TX | LEN=5 | 24 52 02 20 02 | Manual I2C read 0x20, count 2
min POT → 2026-11-12 12:03:36.929 | COM5 | RX | LEN=6 | 24 52 03 00 00 00 | I2C read successful: 00 00 | PIC A

Figure 4-4 GUI Requests ADC from PIC

* for this to work you must write the pointer Packet before Reading PortB or the ADC.

ART 11/12/26

4. Second I²C device. Connect the ADS1115 & PIC slave at the same time. 11/12/26
Probe each at its verified address, From the host, select & read ADS1115 Config & conversion registers using separate write (57) & read (52) call, reusing Part 2 setup if needed. change the ADS1115 input voltage. Confirm you can switch between devices without changing the bridge (master) firmware.

* Process for checking Config Reg in GUI
~~Write 01 C5 83 to ADS1115 address~~
~~Read~~
Write Config pointer 0x01
Write Config data 01 C5 83
write Conversion pointer 00
Read Conversion @ Conversion pointer 00 Length = 2

Communication History

2026-11-12 12:41:16.412 | COM5 | RX | LEN=4 | 24 57 01 00 | I2C write successful
2026-11-12 12:41:24.060 | COM5 | TX | LEN=5 | 24 52 02 49 02 | Manual I2C read 0x49, count 2
2026-11-12 12:41:24.087 | COM5 | RX | LEN=6 | 24 52 03 00 2D 8C | I2C read successful: 2D 8C ← 1st read
2026-11-12 12:41:44.641 | COM5 | TX | LEN=5 | 24 57 02 49 01 | Manual I2C write 0x49, data bytes 1
2026-11-12 12:41:44.677 | COM5 | RX | LEN=4 | 24 57 01 00 | I2C write successful
2026-11-12 12:41:49.136 | COM5 | TX | LEN=7 | 24 57 04 49 01 C5 83 | Manual I2C write 0x49, data b
2026-11-12 12:41:49.176 | COM5 | RX | LEN=4 | 24 57 01 00 | I2C write successful
2026-11-12 12:41:56.262 | COM5 | TX | LEN=5 | 24 57 02 49 00 | Manual I2C write 0x49, data bytes 1
2026-11-12 12:41:56.293 | COM5 | RX | LEN=4 | 24 57 01 00 | I2C write successful
2026-11-12 12:42:00.414 | COM5 | TX | LEN=5 | 24 52 02 49 02 | Manual I2C read 0x49, count 2
2026-11-12 12:42:00.449 | COM5 | RX | LEN=6 | 24 52 03 00 2E A7 | I2C read successful: 2E A7 ← 2nd read

Figure 4-5 Read Conversion register process

5. Limits & errors. Use the GUI to try zero-length, one-byte, ordinary, maximum-length & oversized requests, including a zero-byte read, an address-only write, a 23-byte request read (17), a 24 byte read request (18), & a write with 24 data bytes (25-byte payload, above the bridges limit). Record the raw request, reply or timeout, whether the bridge rejected it, & any I²C activity. Test NACK on an unoccupied addresses & restore operation after a UART framing or overrun error as in Lab 8. Out-of-range requests must not cause buffer overflow or I²C activity. Complete a 23-byte transfer only if your slave supports it; do not extend the part3 slave for this test.

Communication History

2026-11-12 13:40:29.676 | COM5 | INFO | LEN=0 | | Verified bridge connected at COM5
2026-11-12 13:40:38.651 | COM5 | TX | LEN=3 | 24 51 00 | Manual UART command 0x51
2026-11-12 13:40:38.695 | COM5 | RX | LEN=5 | 24 45 02 05 02 | BRIDGE ERROR: unknown command ← bad length
2026-11-12 13:40:38.698 | COM5 | ERROR | LEN=0 | | Manual UART command 0x51 failed: BRIDGE ERROR: unknown comm
2026-11-12 13:40:51.227 | COM5 | INFO | LEN=0 | | Boundary test warning: WARNING: Address is reserved or outside ordin
2026-11-12 13:40:51.230 | COM5 | TX | LEN=4 | 24 50 01 00 | Manual I2C probe 0x00 (boundary test)
2026-11-12 13:40:51.268 | COM5 | RX | LEN=5 | 24 45 02 05 03 | BRIDGE ERROR: invalid I2C address ← invalid Address
2026-11-12 13:40:51.269 | COM5 | ERROR | LEN=0 | | Manual I2C probe 0x00 (boundary test) failed: BRIDGE ERROR: invalid
2026-11-12 13:41:20.197 | COM5 | TX | LEN=5 | 24 57 02 21 00 | Manual I2C write 0x21, data bytes 1
2026-11-12 13:41:20.233 | COM5 | RX | LEN=5 | 24 45 02 04 01 | I2C NACK (address or data) ← forced NACK
2026-11-12 13:41:20.238 | COM5 | ERROR | LEN=0 | | Manual I2C write 0x21, data bytes 1 failed: I2C NACK (address or data)

Figure 4-6 Three error tests displayed in GUI

ART
11/12/26

Experiment 10 I²C Communication

6. Complete the host GUI. Add the remaining features below. Use your GUI to repeat the PIC & ATS1113 transactions, scan all addresses, save & replay commands, then test automatic connection & recovery.

Host GUI requirements

Build a graphical application, extending Lab 08 or using your own design:

- **Custom transmission and lengths:** Select an I²C address, probe, read count, or write data. Enter write data as **hex bytes** (for example, `01 41`) or **text** encoded as UTF-8; display the actual bytes and length before sending. Both modes supply the data portion of a framed write, not unframed UART text. Allow users to deliberately test **zero, short, maximum and oversized payload lengths** and read counts, including values outside the bridge's accepted range. Show a warning but do **not** truncate, split, or prevent a deliberate boundary test. Provide an advanced raw-frame mode for testing declared-length mismatches and unsupported commands. Reject input that cannot be represented as bytes.
- **Saved transmissions:** Name, save, select, edit, delete, and retransmit command sequences. Preserve them across application restarts, including address, operation, payload/read count, and hex/text mode.
- **Timestamped history and file logging:** Display and append **every UART transmission and every received reply/error frame** to a log file, including automatic identity checks and address scans. Include timestamp (date and time), direction, port, raw hex bytes, payload length, and decoded response when available. Also log **all errors detected by the GUI or reported by the bridge**, including Lab 08 framing/overrun packets, new I²C error packets, invalid requests, serial exceptions, malformed/incomplete replies and response timeouts. Timestamp disconnection/reconnection events and indicate which request failed.
- **I²C address discovery:** Provide a Scan button that examines **all 128 seven-bit address values**, `00` – `7F`, and shows progress and a list of acknowledged slave addresses in hexadecimal. The ranges `00` – `07` and `78` – `7F` are reserved for special I²C purposes. Mark those values **Reserved** without sending an I²C transaction. For each ordinary address `08` – `77`, send a probe command (`50`) and classify the result as **ACK**, **NACK**, or **communication error**. Do not treat a skipped reserved address as a NACK.
- **Automatic connection:** At startup, search available serial ports at **9600 8N1** and accept the bridge only after the exact identity reply `24 49 02 33 75` to `24 49 00`. Check bridge connectivity approximately **once per second** while connected, show connected/disconnected status and port, and retry discovery approximately every second after communication loss. Reconnect automatically, including if the port name changes. Never send device commands to an unidentified port.
- **Serial coordination:** Allow only **one outstanding command** at a time, including address scans and connectivity checks. Reassemble fragmented UART replies, enforce a response timeout, and keep the GUI responsive. Queue or defer the one-second identity check while another request is active.

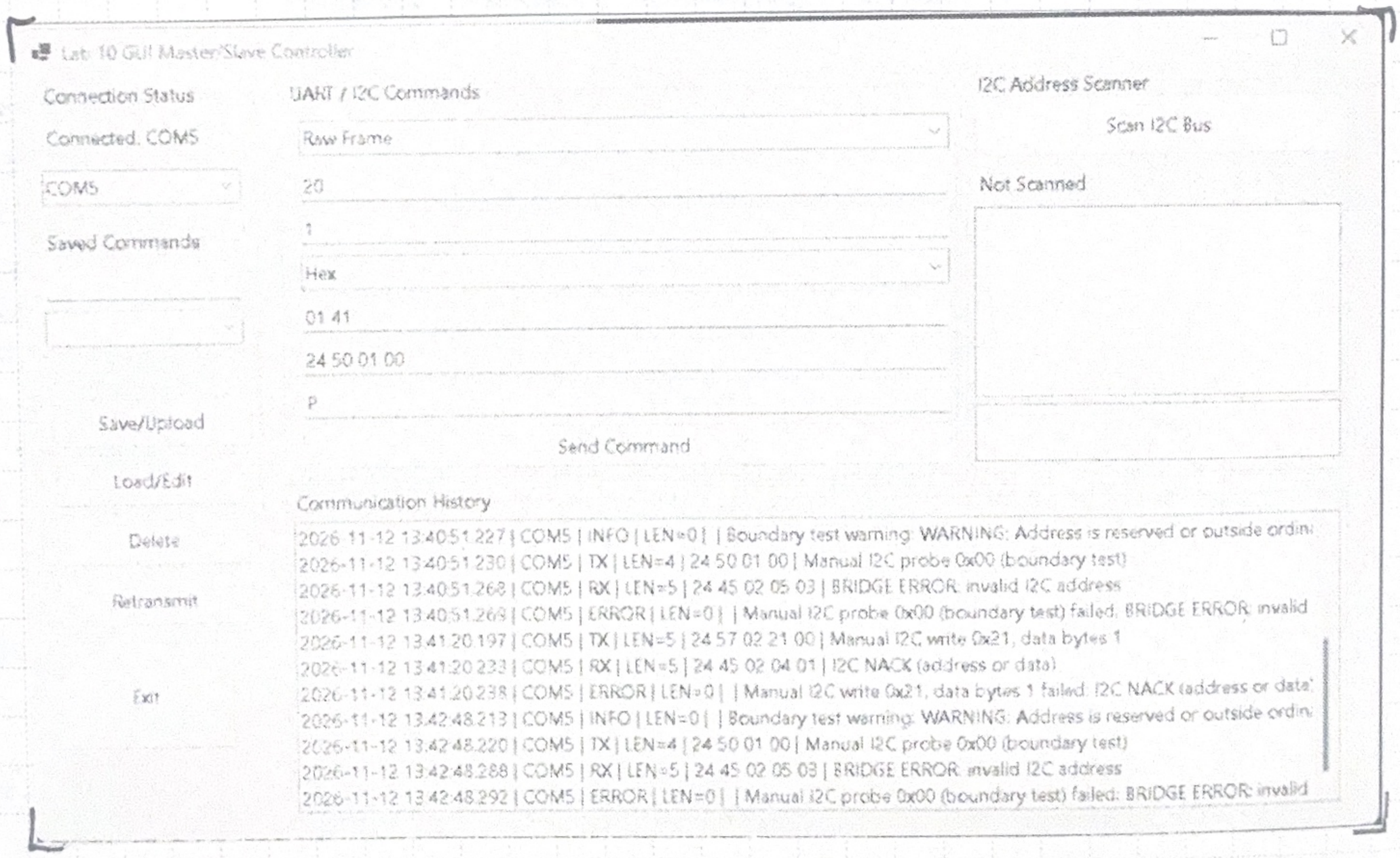


Figure 4-7 full host GUI

Exprirment 10 I^2C Communication

IART
11/12/26

Conclusion

This experiment is centered around I^2C Master-Slave Communication. Essentially this all functions based off of the masters Clock & Data lines. These lines are connected to as many slaves as the part specs could allow based on pull-up resistor calculations. The slaves are driven off the masters clock to maintain timing for proper data transfer. This Lab had us exprirment with multiple slaves, Addresses, & Slave types. We used the slaves to perform the usual functions such as ADC Conversions, Displaying data, & reading pin statuses. We also did a larger test combining UART GUI functionality, treating the Master PIC as a bridge or middle man to control the slave.

IART
11/13/26

IART
11/13/26